

All Closed

设答案集合为 T , 所有 S_i 的交集为 I , 则:

- $T \cap I = \emptyset$, 因为 I 中的元素本来就在所有 S_i 中;
- $T \cup I = T \cup (S_1 \cap \dots \cap S_n) = (T \cup S_1) \cap \dots \cap (T \cup S_n)$ 。由于每个 $T \cup S_i$ 都是线性空间, 而线性空间的交仍是线性空间, 因此 $T \cup I$ 必然对异或封闭。

不妨用线性基维护 $U = T \cup I$, 最后再还原出答案 $T = U - I$ 。记 $\text{span}(S)$ 为 S 张成的线性空间, 即包含 S 的最小异或封闭集合。

显然至少有 $U \supseteq \text{span}(S_i) - S_i$, 于是首先要求出 $\text{span}(S_i) - S_i$ 对 U 贡献了哪些基。为了控制复杂度, 分两种情况:

- 若 $|\text{span}(S_i) - S_i| > \frac{1}{2} |\text{span}(S_i)|$, 则 $\text{span}(S_i) - S_i$ 不可能包含在 $\text{span}(S_i)$ 的真子空间中, 因此它张成的空间就是整个 $\text{span}(S_i)$ 。此时直接将 S_i 逐元素插入线性基;
- 若 $|\text{span}(S_i) - S_i| \leq \frac{1}{2} |\text{span}(S_i)|$, 则 $|\text{span}(S_i)| \leq 2|S_i|$, 可以暴力枚举整个 $\text{span}(S_i)$ 再将其中不属于 S_i 的元素插入线性基。

令 $U_{\text{init}} = \text{span}(I \cup (\text{span}(S_1) - S_1) \cup \dots \cup (\text{span}(S_n) - S_n))$, 已知 $U \supseteq U_{\text{init}}$ 的条件下, $T \cup S_i = U \cup \text{span}(S_i)$, 而两个线性空间的并仍为线性空间当且仅当其中一个包含另一个, 因此只可能有 $U \subseteq \text{span}(S_i)$ 或 $U \supseteq \text{span}(S_i)$ 。

考虑如下算法。初始 $U \leftarrow U_{\text{init}}$, 过程中线性基可能会扩展, 即当发现某个 $U \not\subseteq \text{span}(S_i)$ (此时强制要求 $U \supseteq \text{span}(S_i)$), 就将 S_i 逐元素插入线性基, 容易证明停止扩展时 $T = U - I$ 是唯一且最小的。显然 U 最多扩展 $O(m)$ 次, 如果每次扩展后重新判断整个 U 是否为每个 $\text{span}(S_i)$ 的子空间, 复杂度不优; 正确做法是预先为每个 S_i 建立线性基, 每当 U 新增一个基 b , 只需判断 b 能否被各个还未归为“ $U \supseteq \text{span}(S_i)$ 类”的 S_i 的线性基表示, 这样总时间复杂度为 $O(nm^2 + (\sum c_i)m)$ 。

Bread

设凸多边形周长为 P 。依次绕一周的 n 个顶点后, 凸多边形姿态恢复, 只向右平移 P , 所以滚动很多轮时, 首尾不完整的部分只产生有界影响。若一个周期 $[0, P]$ 内稳定扫过的面积为 B , 所求极限就是 $\frac{B}{P}$ 。

记顶点 A_i 成为支点时在地面上的横坐标为 c_i 。从初始接地边开始累加边长, 即 $c_1 = |A_n A_1|$, $c_i = c_{i-1} + |A_{i-1} A_i|$, 最后 $c_n = P$ 。绕 A_i 旋转时, 顶点 A_j 的轨迹是以 $(c_i, 0)$ 为圆心、 $|A_i A_j|$ 为半径的一段圆弧。需要求面积的图形的上边界就是这若干段圆弧的上包络线横坐标在 $[0, P]$ 以内的部分。

这组圆弧每隔 P 重复。任意两顶点之间的距离不超过边界上较短的一段长度, 故 $R_i \leq \frac{P}{2}$; 计算 $[0, P]$ 时, 只需要前后把圆弧序列前后都倍长一份, 也就是保留圆心横坐标位于 $c_i - P, c_i, c_i + P$ 的圆弧。

可以证明, 有效的圆弧只有以各个支点 $(c_i, 0)$ 为圆心、以 $R_i = \max_j |A_i A_j|$ 为半径的上半圆。证明分为两步: 第一步是把这些圆弧补充成完整的上半圆不会改变包络线; 第二步是半径小的上半圆会被半径大的完全覆盖故无效。

通过分治决策单调性或 SMAWK 算法计算出每个圆心对应的最大半径 R_i , 问题转化为计算 $O(n)$ 个上半圆的上包络线下方面积。注意传统的旋转卡壳求凸多边形直径算法用于计算凸多边形每个顶点距离的最远点是错误的, 现场部分选手因为这个问题而始终没有通过该题。一个五个点的反例:

$$A_1 = (1, 0) \quad A_2 = (5, 2) \quad A_3 = (6, 6) \quad A_4 = (0, 2) \quad A_5 = (0, 0)$$

把上半圆写成 $y^2 = R^2 - (x - c)^2$ 。令 $z = x^2 + y^2$ ，就有 $z = 2cx + R^2 - c^2$ 。固定 x 时，最高的圆等价于对应直线的 z 最大值。按圆心递增加入圆时，直线斜率也递增，可以用单调栈维护上包络。两个相邻圆 (c_1, R_1) 、 (c_2, R_2) 的切换位置为：

$$x = \frac{c_1 + c_2}{2} + \frac{R_1^2 - R_2^2}{2(c_2 - c_1)}$$

若新交点不大于栈顶圆的起始位置，栈顶圆没有获胜区间，直接弹出；否则记录交点并入栈。这样每个圆最多进出栈一次。

计算圆弧下面积可以通过积分手段，也可以通过将圆弧下方割补为一个扇形和两个三角形计算。

总时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

Cut Tree

先按边编号从小到大建立 Kruskal 重构树：原树顶点作为叶子，每加入一条边就把它两侧的连通块合并，并新建一个内部节点。这样边 e 对应的内部节点编号可以设为 $n + e - 1$ ，祖先编号严格大于后代。两个原树顶点 a, b 的 LCA 就对应它们路径上编号最大的边，因为这正是两点在加边过程中第一次连通的时刻。

按重构树叶子的顺序记录相邻叶子的 LCA。若两个叶子的序号为 $a < b$ ，则它们的 LCA 是区间 $[a, b)$ 中相邻叶子 LCA 的最大值，因此对这个数组建立 ST 表后即可 $O(1)$ 查询任意 LCA。

取一个块长 B ，先按询问左端点所在的块分组，再保证每组至多有 B 个询问。对一组询问，记左端点的最小值和最大值为 L, R ，右端点最大值为 H 。把编号在 $[L, R]$ 内的边，以及本组询问中所有编号大于 R 的待删除边，统称为特殊边。特殊边数量为 $O(B)$ ，其余可能出现的边构成公共森林。于是每个询问都可以看成：取公共森林中编号不超过 r 的部分，再补入满足 $l \leq e \leq r$ 且 $e \neq f$ 的特殊边。

先考虑只由公共边组成的森林。把原树定根后，按父亲先于孩子的顺序扫描即可求出公共森林的连通块。对其中一个连通块，任意两点之间的原树路径都没有被删掉，所以它们在加入公共边时的连通时刻仍然是路径上编号最大的边。于是，全局重构树在这个连通块的叶子上的虚树，正好记录了该连通块随 r 增大时的合并过程。按全局叶序扫描，并为每个公共连通块维护一个栈，就能在线性时间内构造这些虚树；每个节点只会入栈、出栈一次。

特殊边的两个端点称为关键点。对虚树进行压缩时，只保留包含关键点部分：若一个内部节点的两侧都没有关键点就忽略，只有一侧有关键点就沿用这一侧，两侧都有关键点时新建一个压缩节点并合并两边。关键点数为 $O(B)$ ，所以压缩树的大小也是 $O(B)$ 。将本组询问按 r 递增处理，随着 r 扫描公共重构树的合并事件，并维护公共森林当前的最大连通块权值。

回答某个询问时，先在压缩树上倒序扫描，得到每个关键点当前所在的公共连通块。再把特殊边统一定向为原树中的父亲到孩子，并按孩子深度从深到浅处理需要保留的特殊边。若特殊边为 (u, v) ，直接把子连通块的权值加到父连通块上。从深到浅处理保证一条边下面的其他特殊边都已经处理完。公共部分的最大值和这些累加过程中出现的最大值的较大者，就是答案；点权为正保证了连通块合并后只会变大。

一次预处理的复杂度为 $O(n \log n)$ 。每组构造公共森林和压缩树需要 $O(n)$ ，每个询问处理 $O(B)$ 条特殊边，因此总复杂度为：

$$O\left((n + q) \log n + \frac{n(n + q)}{B} + qB\right)$$

取 B 为相应的根号级块长即可得到 $O((n + q)\sqrt{n})$ ，空间复杂度为 $O(n \log n + q)$ 。

Divide and Conquer

令叶子数为 $N = 2^n$ 。因为每个节点的值等于它下面叶子中的最大值，所以约束 (u, x) 的意思是：值 x 要放在 u 的子树里，而比 x 大的值不能放在里面。

把根节点的约束固定为 $(1, N)$ ，然后只保留有约束的节点，并把每个节点连到最近的有约束祖先，忽略中间没有约束的节点。下面几种情况直接无解：同一个节点被要求不同的值；父节点的要求值小于子节点；同一个值出现在两个互不包含的子树中，因为一个值只会出现在一个叶子上。

对压缩树中的每个节点，删掉它下面直接有约束的子树，剩下的位置称为这个节点的“自由位置”。自由位置的数量就是当前子树的叶子数减去这些子树的叶子数，所有叶子位置恰好被分到一个区域。若节点要求的值是 x ，这个区域里只能放不超过 x 的数。对于值 x ，找到要求值为 x 的最下面那个节点； x 必须放在它的自由位置中，否则就无法同时满足所有关于 x 的要求。这个区域没有位置时也直接无解。

最后从 N 到 1 依次放数，并维护一个可用位置池。处理到 x 时，把要求值为 x 的区域加入位置池；如果 x 没有被要求，就从池中任选一个位置，方案数是池的大小；如果 x 有要求，就只能从刚找到的最下面节点的区域中选，方案数是该区域的大小；放好后删去一个位置。某一步没有可选位置就无解，所有数放完后答案就是这些方案数的乘积。时间复杂度为 $O((q + 2^n)n)$ ，空间复杂度为 $O(q + 2^n)$ 。

Exponent

当 $n > 1$ 时，若 $\gcd(a, n) > 1$ ，则 $a^x \equiv 1 \pmod{n}$ 不可能成立；而 $\gcd(a, n) = 1$ 时由欧拉定理一定存在这样的 x 。所以只有与 n 互质的 a 需要计数，其余数的贡献都是 0。 $n = 1$ 时唯一的数 $a = 1$ 的阶为 1，答案也是 1。

设 $n = \prod p^e$ 。由中国剩余定理，一个与 n 互质的数可以唯一地看成分别模各个 p^e 的互质数；它模 n 的阶就是这些局部阶的最小公倍数。对每个质因数幂分别处理：

- 若 p 是奇素数，取模 p^e 的原根 g 。令 $m = (p - 1)p^{e-1}$ ，则所有互质数恰好是 g^r ($0 \leq r < m$)，对应的阶为 $\frac{m}{\gcd(m, r)}$ 。
- 若 $p = 2$ ， $e = 1$ 时只有一个互质剩余，不需要参数； $e = 2$ 时可以用 3^r ($0 \leq r < 2$) 表示，得到一个长度为 2 的参数； $e \geq 3$ 时每个奇数都能唯一写成 $(-1)^\varepsilon 5^r$ ，其中 $\varepsilon \in \{0, 1\}$ 且 $0 \leq r < 2^{e-2}$ ，两个部分的阶取最小公倍数，因此得到长度为 2 和 2^{e-2} 的两个参数。

于是只需收集一系列参数长度 $m_1, m_2, \dots$ ：每个奇素数幂贡献 $(p - 1)p^{e-1}$ ， 2^1 不贡献， 2^2 贡献 2， 2^e ($e \geq 3$) 贡献 2 和 2^{e-2} 。对每个参数选择指数 $r_i \pmod{m_i}$ 后，就唯一确定了一个候选数，而且第 i 个局部参数贡献的阶为 $\frac{m_i}{\gcd(m_i, r_i)}$ 。

把每个 m_i 分解为素数幂。固定一个素数 q ，对每个满足 $e_i = v_q(m_i) > 0$ 的参数 ($v_q(m_i)$ 表示 q 在 m_i 中的指数)，只观察指数 r_i 的 q 部分。由于不同素数部分可以由中国剩余定理独立选择，整体阶中 q 的指数就是各个参数贡献的 q 指数的最大值。若要求这个最大值不超过 t ，对于一个含有 q^{e_i} 的参数，必须有 $q^{\max(e_i - t, 0)} \mid r_i$ ，所以该参数有 $q^{\min(t, e_i)}$ 种选择。于是只看 q 部分时，满足条件的选择数为：

$$F_q(t) = q^{\sum_i \min(t, e_i)} \quad (t \geq 0)$$

恰好使整体阶中 q 的指数等于 t 的选择数为 $F_q(t) - F_q(t - 1)$ 。因此所有选择对 q 部分阶之和的贡献为：

$$S_q = 1 + \sum_{t=1}^{\max e_i} q^t (F_q(t) - F_q(t - 1))$$

不同素数的指数部分相互独立，而整体阶是它们的乘积，所以最终答案就是各个 S_q 的乘积。

预处理不超过 31623 的所有素数后，试除分解 n 以及各个 m_i 即可。若用 $\omega(n)$ 表示 n 的不同质因子个数，则每组数据至多需要分解 $O(\omega(n))$ 个不超过 n 的数，试除部分的复杂度为 $O((\omega(n) + 1)\pi(\sqrt{n}))$ 。

Folding Game of Ohto Ai

可以证明无论怎么操作，操作次数一定相同。证明方法考虑，虽然不同的操作得到的树可能不同构，操作后形态不同的子树一定会在影响答案前变成形态相同的子树。

但是暴力操作的时间复杂度是 $O(N^2)$ 的。考虑如果当前直径有奇数条边，则一次操作会将直径长度减少；如果当前直径有偶数条边，则一定存在一个中心点，这个中心点有一些深度为直径一半的子树，一定会把这些子树合并到同一个子树内，才会使得当前直径变短，所以可以一轮操作同时合并所有以中心点为根，深度为直径一半的子树。

由于一次操作会减少的点的数量跟直径长度同阶，所以整个操作流程不同的直径长度数量是 $O(\sqrt{N})$ 级别的。对于每个相同的直径一轮操作就会使得直径变短，一轮操作的复杂度是 $O(N)$ ，综合复杂度为 $O(N\sqrt{N})$ 。

没有特意卡 $O(N\sqrt{N} \log N)$ ，瓶颈在于排序去重或者并查集。

Bonus：本题有一个更深入的性质是，固定一个初始的直径端点，每次折叠都折向它，则它一直都会是直径端点。然后用可并堆维护这个折叠过程，复杂度 $O(N \log N)$ 。

Ghost of Tsushima

[更详细的题解](#)

设选出的区间数量为 m ，并令 d 表示跨过边 $(n, 1)$ 的区间数量。把所有左端点从小到大记为 $l_1 < \dots < l_m$ ，右端点从小到大记为 $r_1 < \dots < r_m$ 。沿环依次观察左端点时，对应右端点也必须保持相同的循环顺序，否则会有两个区间的左右端点次序相反，其中一个区间包含另一个。因此配对只能是某个循环位移；又因为恰有 d 个区间跨过边 $(n, 1)$ ，所以只能把 l_i 配给下标为 $i + d$ (超过 m 后循环) 的右端点。于是只需要枚举左右端点是否出现，以及 d 。

令 $c_L(i), c_R(i)$ 分别表示点 i 是否为左端点、右端点，令 P_L, P_R 为它们的前缀和，并记 $D(i) = P_L(i) - P_R(i)$ 。先把跨过边 $(n, 1)$ 的 d 个区间视为已经打开，从左到右扫描时，每遇到一个左端点就多一个打开的区间，每遇到一个右端点就少一个。因此边 $(i, i + 1)$ 的覆盖数是 $d + D(i)$ ，点 i 的覆盖数是 $d + P_L(i) - P_R(i - 1) = d + D(i) + c_R(i)$ 。把这两类数交错排成序列 Y ，即 $Y_{2i} = D(i) + c_R(i)$ 、 $Y_{2i+1} = D(i)$ 。按照上面的唯一配对，所有前缀中打开的区间数均在 $[0, m]$ 内，恰好等价于每个左端点都和正确一侧的右端点配对，所以区间集合合法当且仅当所有 $d + Y_i$ 都在 $[0, m]$ 内。

固定 m 时，左端点和右端点各自选出 m 个位置，因此 (c_L, c_R) 一共有 $\binom{n}{m}^2$ 种。

再定义 $Q_{2i} = 2Y_{2i}$ 、 $Q_{2i+1} = 2Y_{2i+1} + 1$ 。由 $D(i) = D(i - 1) + c_L(i) - c_R(i)$ 可得相邻两步分别为 $Q_{2i} - Q_{2i-1} = 2c_L(i) - 1$ 和 $Q_{2i+1} - Q_{2i} = 1 - 2c_R(i)$ ，所以每一步都是 1 或 -1；又因为左右端点总数相同， Q 走 $2n$ 步后回到起点。反过来，两类步长的正负也能唯一还原 c_L, c_R ，所以端点序列与这种闭合格路一一对应。把 Q 整体上移 $2d$ 后，覆盖次数限制就变成一条不能越过上下边界的格路限制。

简要说明这里使用的反射原理。设两条禁线之间的距离为 L ，固定起点高度 $a \in [1, L - 1]$ ，走 $2n$ 步回到 a 且不碰禁线的路径数为：

$$N_a = \sum_{j \in \mathbb{Z}} \left(\binom{2n}{n+jL} - \binom{2n}{n+a+jL} \right)$$

将一条闭合路径的第一步循环移到末尾，会把起点高度的奇偶性翻转，这是合法路径间的双射，因此只对所需奇偶性的起点求和，结果是所有 $a = 1, \dots, L - 1$ 的一半。记 $A = \sum_j \binom{2n}{n+jL}$ ，对所有 a 求和时，第二项恰好遍历除下标模 L 与 n 同余以外的所有二项式系数，其和为 $2^{2n} - A$ ，故所求路径总数为：

$$\frac{L}{2} \sum_{j \in \mathbb{Z}} \binom{2n}{n+jL} - 2^{2n-1}$$

这里的求和实际上只有有限个二项式系数非零。对于边覆盖，限制 $0 \leq d + D(i) \leq k$ 等价于整条格路满足 $0 \leq 2d + Q_i \leq 2k + 2$ ，也就是避开高度 -1 和 $2k + 3$ 两条禁线，其间距为 $L = 2k + 4$ ，所以初步计数为：

$$(k+2) \sum_j \binom{2n}{n-2j(k+2)} - 2^{2n-1}$$

对于点覆盖，限制 $0 \leq d + Y_i \leq k$ 等价于 $0 \leq 2d + Q_i \leq 2k + 1$ ，两条禁线的间距为 $L = 2k + 3$ ，所以初步计数为：

$$\frac{2k+3}{2} \sum_j \binom{2n}{n+j(2k+3)} - 2^{2n-1}$$

当 $m > k$ 时，覆盖次数限制已经蕴含合法性所需的上界 m ，初步计数没有多算；只需修正 $m \leq k$ 的情况。固定 (c_L, c_R) 后，合法条件给出 $-\min Y \leq d \leq m - \max Y$ ，所以合法的 d 有 $(m+1) - (\max Y - \min Y)$ 个。点覆盖的初步计数把右端放宽为 $k - \max Y$ ，会计入 $(k+1) - (\max Y - \min Y)$ 个 d ，因此每个端点序列恰好多算 $k - m$ 次，总共需要减去 $\sum_{m=0}^k (k-m) \binom{n}{m}^2$ 。

边覆盖的初步计数使用的是 D 的范围，因而计入了 $(k+1) - (\max D - \min D)$ 个 d ，减去实际合法数量后，每个端点序列多算 $k - m + \min D - \min Y + \max Y - \max D$ 次。因为 $c_R(i) \geq 0$ ，有 $\min Y = \min D$ 。再把右端点序列循环右移一格，记为 $c'_R = (c_R(n), c_R(1), \dots, c_R(n-1))$ ，对应的前缀差为 D' ，则 $D'(i) = Y_{2i} - c_R(n)$ 。又因为 Y 的最大值一定能在某个 Y_{2i} 处取得，所以 $\max D' = \max Y - c_R(n)$ 。循环平移是所有右端点序列上的双射，因此对全部端点序列求和后， $\max D' - \max D$ 的贡献相消，只剩下 $\sum c_R(n)$ 。因此边覆盖的总多算次数为：

$$\sum_{m=0}^k \left((k-m) \binom{n}{m}^2 + \binom{n}{m} \binom{n-1}{m-1} \right) = \sum_{m=0}^k \left(k - \frac{n-1}{n} m \right) \binom{n}{m}^2$$

从初步计数中减去这些重复项，得到两类答案（约定下标越界的二项式系数为 0）：

$$\text{Ans}_{\text{edge}}(k) = \sum_{m=0}^k \left(\frac{n-1}{n} m - k \right) \binom{n}{m}^2 + (k+2) \sum_j \binom{2n}{n-2j(k+2)} - 2^{2n-1}$$

$$\text{Ans}_{\text{point}}(k) = \sum_{m=0}^k (m-k) \binom{n}{m}^2 + \frac{2k+3}{2} \sum_j \binom{2n}{n+j(2k+3)} - 2^{2n-1}$$

预处理阶乘和逆阶乘即可 $O(1)$ 计算二项式系数，并用前缀和递推维护关于 m 的两类和。公式中的除法均在模 998244353 下用逆元实现。对固定的 k ，第二个求和只有 $O(n/k)$ 项非零，因此枚举全部 $k = 1, \dots, n$ 的总时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

Hidden Track (Easy Version)

[更详细的题解](#)



令 $k = \lceil \log_2 n \rceil$, 第 j 位直接使用掩码 $m = 2^j$, 其中 $0 \leq j < k$ 。这样每次询问只观察顶点编号的第 j 位, 所有位合起来可以区分所有顶点。

先对每个 j 查询一次不翻转任何点的答案, 记为 b_j 。再查询把顶点 v 翻转后的答案, 记为 $q_j(v)$ 。设:

$$f_j(x) = \begin{cases} 1 & x \text{ 的第 } j \text{ 位为 } 0 \\ -1 & x \text{ 的第 } j \text{ 位为 } 1 \end{cases}$$

翻转 v 只会改变与 v 相连的边。对一条相邻边 (v, u) , 若 v, u 的第 j 位相同, 这条边会从不跨集合变为跨集合, 回答增加 1; 若这一位不同, 回答就减少 1。这两种变化恰好都是 $f_j(v)f_j(u)$, 对 v 的所有邻边求和便有:

$$q_j(v) - b_j \equiv f_j(v) \sum_{u \sim v} f_j(u) \pmod{3}$$

由于 $f_j(v)^2 = 1$, 令 $s_j(v) = f_j(v)(q_j(v) - b_j) \pmod{3}$, 就得到 v 的所有邻居在第 j 位上的 f_j 值之和。

如果 v 是端点, 它只有一个邻居, 所以 $s_j(v)$ 只能是 1 或 2: 分别对应邻居这一位为 0 或 1。如果 v 是内部点, 它有两个不同的邻居; 只要找一位使两个邻居不同, 这一位的和就为 0。于是当 $n \geq 2$ 时, v 是端点当且仅当所有 $s_j(v)$ 都不为 0。

确定一个端点后即可沿路径恢复。若当前点 v 的前一个点是 `pre`, 那么对每一位都有:

$$f_j(\text{next}) \equiv s_j(v) - f_j(\text{pre}) \pmod{3}$$

右侧一定会得到 1 或 -1, 从而逐位确定下一个顶点的编号。第一个点没有前驱, 直接使用端点规则恢复唯一邻居。

为了不超过 nk 次询问, 先按编号从小到大检查顶点, 直到找到第一个端点 a 。每个已检查顶点的 $q_j(v)$ 结果都缓存下来, 然后从 a 开始沿路径推导; 遇到尚未询问的点再补做 k 次查询。由于题目保证排列首尾满足 $p_1 < p_n$, 按编号找到的第一个端点就是应当输出的起点, 而另一个端点编号更大, 在寻找 a 时不会被询问。沿路径恢复到已经得到全部 n 个点时直接停止, 也不需要最后一个端点的信息。因此只会对 $n - 1$ 个顶点各查询 k 次, 再加上 k 次基准查询, 总次数为 $(n - 1)k + k = nk$ 。 $n = 1$ 时直接输出唯一的排列即可。

每个顶点的每一位只处理常数次数, 时间复杂度为 $O(nk)$, 空间复杂度为 $O(nk)$ 。

Island

设节点 u 的子树中最左和最右的叶子分别为 L_u, R_u 。子树内的叶子在 DFS 序中连续，因此处理完这棵子树后，它只可能通过 u, L_u, R_u 与外部相连。于是只需记录这三个端点的连通状态：三点全连、只有 u, L_u 相连、只有 u, R_u 相连、只有 L_u, R_u 相连、三点互不相连，共五种；端点重合时先去重即可。

对每种状态维护方案数，以及这些方案中已经完全离开三个端点的连通块数之和。合并两部分时枚举它们的状态，得到合并后三个端点的状态，并统计有多少个连通块在这次合并后不再接触任何端点。由于状态只有五种，可以预先枚举所有会用到的合并方式，用一个很小的并查集求出新状态和新产生的连通块数，之后转移时直接查表即可。若新产生了 d 个连通块，则方案数相乘，连通块总数在两边原有贡献之外再加上 d 乘对应的方案数。

按照后序遍历做树形 DP。叶子的状态只有它自己；处理非叶节点 u 时，按照输入顺序从左到右加入儿子。加入一个儿子时，需要同时考虑树边 (u, v) 是否保留；从第二个儿子开始，还要考虑前一个儿子的最右叶与当前儿子的最左叶之间的链边是否保留。每加入一个儿子，仍只保留当前部分的 u 、最左叶和最右叶作为端点。处理完根后去掉全部端点，剩余连通块全部计入答案。若叶子数为 k ，总边数为 $m = n + k - 2$ ，最后将连通块数之和除以 2^m 即可。转移次数和每次转移的状态数都是常数，时间复杂度为 $O(n)$ 。

也可以利用平面图欧拉公式来做。固定这张图自然的平面嵌入，设保留子图的边数为 E ，由基本环围出的有界面数为 F ，连通块数满足 $C = n - E + F$ 。其中 $\mathbb{E}[E] = m/2$ ，因此只需计算 F 的期望。合并两部分或加入一条边时，根据端点是否已经连通判断是否新形成一个有界面，并累计基本环的数量。这个做法和上面的 DP 基本相同，同样需要记录 u, L_u, R_u 三个端点的五种连通状态，并同样可以预处理所有转移。

Joker or Judger

签到题，由于数据范围较小，你可以使用任意你喜欢的方式通过本题。

一个可行的实现方式是，把所有需要匹配的字符串和对应的分数放进 `std::map` 里（包括带 X 的字符串可以拆成 12 个字符串塞进 `std::map`），然后直接求和与 P 比较大小。

另外就像题目提示的那样，你可以直接在样例中复制所有的对应字符串。

K-MEX

我们设原数组 a 的 $\text{mex}(a)$ 为 x 。我们发现，对于任意给定的 k ，一定有 $k - \text{mex}(a) \geq x$ 。

根据 mex 的定义，如果要想让答案超过 x ，那么 x 一定出现在数组里。这样的 k 至多只有 n 种可能，分别为 $x + a_i$ ，其中 $1 \leq i \leq n$ 。

我们先枚举这样的 k 。对于一个固定的 k ， a_i 只会变成 a_i 或者 $k - a_i$ 两种。我们令 $u = \min(a_i, k - a_i)$ ， $v = \max(a_i, k - a_i)$ 。如果数组中没有 u ，那么我们就让 $a_i = u$ ，否则让 $a_i = v$ 。然后我们将答案记忆化。询问的时候可以用 $O(1)$ 或者 $O(\log n)$ 单次查询。

时间复杂度 $O(n^2 + q)$ 或者 $O(n^2 + q \log n)$ 。

Loop

格子 (i, j) 中的数是 $a_{(i-j) \bmod n}$ 。把原序列循环地向左右展开，向下和向右就分别相当于在序列上向右和向左走一步。因此原问题等价于从 a_0 出发，向左、向右各走 $n - 1$ 步，最后回到 a_0 。

可以猜测，存在一种最优走法只走 a_0 的某一侧，并在到达最远位置后，不断与前一个位置反复横跳，最后再回到 a_0 。

把相邻位置的权值之和记作边的代价，设这一侧从近到远的边代价为 $c_1, c_2, \dots$ 。若最远走到第 k 条边，前 $k - 1$ 条边各往返一次，剩余的 $n - k$ 次往返都在第 k 条边上，代价就是 $a_0 + \sum_{i=1}^{k-1} c_i + (n - k)c_k$ 。

分别从两个方向扫描所有 k ，维护边代价的前缀和即可。时间复杂度为 $O(n)$ 。

简单证明这个结论。任意合法游走在每条边上向两个方向经过的次数相同。选择其中代价最小的一条边，保留从 a_0 到这条边的路径各往返一次，再把其余所有往返都移到这条边上。由于选择的是最小代价，答案不会变大，于是它可以作为最远的边，所有多余步数也都在这里反复横跳。